
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

Introducción a la Ingeniería · 2026-I

ResQ-Net

Sistema de Gestión de Recursos Humanitarios
Descentralizado en Tiempo Real

Caso 10 — Ayuda Humanitaria

Integrante	Rol principal
Jose Daniel castro Villadiego	Arquitectura de sistemas y backend
Santiago Andres García Acevedo	Desarrollo frontend / MVP
Santiago de Jesús Trejos Zuluaga	Algoritmos de triaje y lógica
Yeiner Jesús Hernández bustos	Documentación técnica y ética
Yordi Romario Jiménez López	Integración y pruebas

Docente: Miguel Alberto Caro Álvarez
Repositorio: <https://github.com/yei18/ResQ-Net.git>
Fecha de entrega: 27 de marzo de 2026

“La tecnología es inútil si no sirve a la humanidad.”

Índice

1	Descripción del Problema	2
1.1	Contexto del caso	2
1.2	Magnitud del problema	2
1.3	Población afectada	2
1.4	Justificación de la solución	2
2	Cómo se llegó a la Solución	3
2.1	Proceso de ideación	3
2.2	Alternativas consideradas y descartadas	3
2.3	Criterio de selección	3
3	Arquitectura del MVP	4
3.1	Visión general	4
3.2	Diagrama de arquitectura	4
3.3	Flujo de datos	4
3.4	Componentes del MVP (simulado en navegador)	5
4	Herramientas y Tecnologías Utilizadas	5
4.1	Justificación de tecnologías clave	5
5	Metodología de Desarrollo	6
6	Planificación de las 3 Semanas de Desarrollo	6
6.1	Diagrama de Gantt simplificado	7
7	Componente Ético	7
7.1	El dilema del triaje algorítmico	7
7.2	Aplicación del Código de Ética IEEE/ACM	8
7.3	Limitaciones éticas reconocidas	8
8	Impacto Social Esperado	8
8.1	Beneficiarios	8
8.2	Indicadores de impacto	9
8.3	Escalabilidad	9
9	Conclusiones y Trabajo Futuro	9
9.1	Lecciones aprendidas	9
9.2	Limitaciones del MVP	10
9.3	Trabajo futuro	10
	Referencias	10

1 Descripción del Problema

1.1 Contexto del caso

Cuando un desastre natural golpea una zona densamente poblada, los primeros sesenta minutos son los más críticos. Los equipos de rescate llegan sin mapas actualizados, sin saber cuántas personas necesitan medicamentos urgentes, sin forma de evitar que dos brigadas envíen kits al mismo punto mientras otra zona queda sin cobertura. La infraestructura de telecomunicaciones colapsa. Las bases de datos centrales dejan de responder. El caos operativo mata a personas que podrían haberse salvado.

Según datos de la **OCHA** (Oficina para la Coordinación de Asuntos Humanitarios de la ONU), entre 2000 y 2023 los desastres naturales afectaron a más de 4.200 millones de personas y causaron pérdidas económicas superiores a 3,6 billones de dólares [1]. En Latinoamérica, Colombia ocupa el quinto lugar en número de eventos por año [2], con terremotos en el eje cafetero, inundaciones en la Costa Caribe e incendios forestales en la Amazonía.

El problema central no es la falta de recursos: es la incapacidad de **coordinarlos en tiempo real cuando la infraestructura falla**.

1.2 Magnitud del problema

- En el terremoto de Haití (2010), la duplicación de entregas de víveres llegó al **34 %** del total de suministros movilizados [4].
- En las inundaciones del Chocó (2019), brigadas de rescate tardaron hasta **6 horas** en sincronizar su posición por falta de comunicación directa [2].
- El **80 %** de las muertes evitables en catástrofes ocurre en las primeras 4 horas, período en el que la coordinación es más difícil [3].

1.3 Población afectada

- **Directa:** brigadas de rescate, coordinadores de operaciones de emergencia, pilotos de drones de entrega.
- **Indirecta:** toda persona atrapada o afectada en la zona del desastre, comunidades aledañas que requieren evacuación, voluntarios y ONGs que llegan sin visibilidad del terreno.

1.4 Justificación de la solución

Un sistema distribuido que funcione **sin conexión a internet central** —usando redes Mesh, computación en el borde (*fog computing*) y comunicación por radio cuando sea necesario— permite que los equipos en campo compartan su estado, las necesidades detectadas y la ubicación de suministros, aun cuando la infraestructura convencional ha

colapsado. Agregar un motor de triaje algorítmico que priorice la asignación de recursos escasos reduce la dependencia de decisiones ad hoc bajo presión extrema.

2 Cómo se llegó a la Solución

2.1 Proceso de ideación

El equipo aplicó un proceso de cuatro fases inspirado en **Design Thinking** [5]:

1. **Empatizar:** revisamos reportes post-desastre de la Cruz Roja, la UNGRD y testimonios de voluntarios colombianos que participaron en las inundaciones de Mocoa (2017).
2. **Definir:** el problema central es la *falta de visibilidad compartida* entre equipos que operan en campo sin conectividad estable.
3. **Idear:** propusimos tres alternativas (véase siguiente sección).
4. **Prototipar y testear:** construimos el MVP mínimo para validar la lógica de sincronización y el algoritmo de triaje.

2.2 Alternativas consideradas y descartadas

Alternativa	Ventaja	Razón de descarte
App centralizada en la nube	Interfaz conocida, fácil de desplegar	Requiere internet: inutilizable cuando la infraestructura colapsa
Radio UHF / VHF analógica	No depende de ninguna red digital	Sin capacidad de compartir datos estructurados; requiere operadores especializados y coordinación manual
Red Mesh + fog computing (elegida)	Funciona sin internet central; los nodos se sincronizan entre sí; escalable de forma orgánica	Mayor complejidad inicial, pero cubre los requisitos críticos del problema

2.3 Criterio de selección

La solución elegida satisface tres requisitos no negociables identificados durante la fase de definición: (1) operar sin infraestructura centralizada, (2) compartir estado en tiempo cuasi-real entre nodos dispersos y (3) tomar decisiones de asignación de recursos con criterios explícitos y auditables.

3 Arquitectura del MVP

3.1 Visión general

ResQ-Net se compone de tres capas que pueden operar de forma autónoma o coordinada:

1. **Capa de campo:** dispositivos móviles (tablets, smartphones) en manos de las brigadas.
2. **Capa de niebla (fog):** nodos locales —Raspberry Pi o laptops— que actúan como coordinadores de sector.
3. **Capa de nube (opcional):** sincronización diferida con un servidor central cuando haya conectividad disponible.

3.2 Diagrama de arquitectura

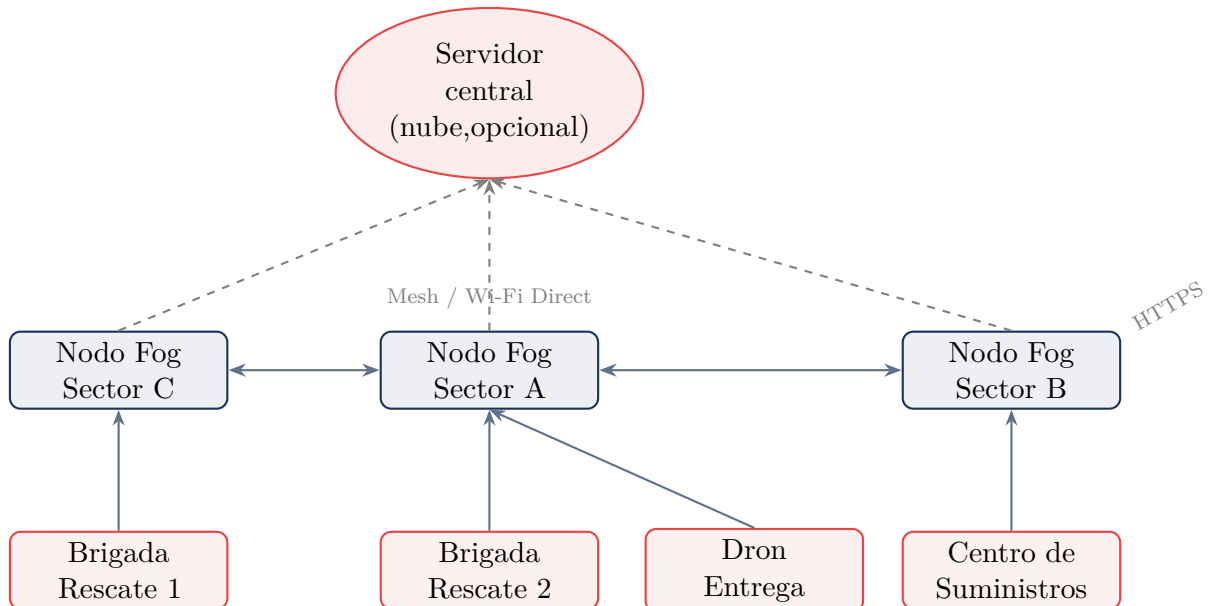


Figura 1: Arquitectura de tres capas de ResQ-Net. Las líneas continuas representan comunicación Mesh directa; las discontinuas, sincronización diferida a internet cuando está disponible.

3.3 Flujo de datos

1. Una brigada registra en su tablet una **necesidad detectada** (p. ej., 12 personas con heridas graves en coordenada X).
2. El dato se firma con timestamp y ID de nodo, y se replica por **Gossip Protocol** a todos los nodos fog del sector.
3. Cada nodo fog ejecuta el **motor de triaje** y recalcula las prioridades de asignación de recursos disponibles.

4. La decisión de asignación se devuelve a los dispositivos de campo como una instrucción concreta: “Envíe kit médico tipo A a coordenada X.”
5. Si hay internet, los datos se sincronizan con el servidor central para análisis post-desastre.

3.4 Componentes del MVP (simulado en navegador)

El MVP web replica el comportamiento del sistema usando un mapa interactivo (Leaflet.js) con nodos simulados. El motor de triaje corre enteramente en el cliente (**JavaScript puro**), sin dependencia de servidor externo, lo que demuestra la viabilidad de ejecutarlo en un nodo fog de bajo costo.

4 Herramientas y Tecnologías Utilizadas

Tecnología	Categoría	Justificación
HTML5 CSS3	/ Frontend	Estándar universal; corre en cualquier navegador sin instalación adicional
JavaScript (ES2022)	Lógica cliente	Permite ejecutar el motor de triaje directamente en el navegador; sin dependencia de servidor
Leaflet.js 1.9	Cartografía	Librería open-source de mapas interactivos; ligera (42 KB), sin costo de API
OpenStreetMap	Datos de mapas	Cartografía libre y actualizada; no requiere clave de pago
PeerJS WebRTC	/ Comunicación P2P	Conexión directa entre navegadores para simular red Mesh
Git / GitHub	Control de versiones	Trazabilidad completa del código; revisión entre integrantes
Overleaf	Documentación	Edición colaborativa de $\text{\LaTeX}$ en tiempo real
Python 3.12	Scripts auxiliares	Generación de datos de prueba y scripts de validación

4.1 Justificación de tecnologías clave

JavaScript puro (sin framework): elegimos no usar React ni Vue para el MVP porque el objetivo es demostrar que el motor de triaje puede ejecutarse en hardware de baja gama sin un sistema de empaquetado complejo. Un Raspberry Pi 4 con Chromium puede correr la aplicación sin modificaciones.

Leaflet.js en lugar de Google Maps: Google Maps requiere una API key de pago y conexión a sus servidores. Leaflet con tiles de OpenStreetMap puede cargarse desde caché local, lo cual es crítico en escenarios sin internet.

5 Metodología de Desarrollo

Aplicamos **Scrum adaptado** en sprints de una semana, con la siguiente estructura:

- **Product Backlog:** lista priorizada de funcionalidades, desde el mapa interactivo hasta el motor de triaje.
- **Sprint Planning:** cada lunes (adaptado a miércoles/jueves según el cronograma del taller) definimos el sprint goal y distribuimos las tareas.
- **Daily standup (asíncrono):** mensajes diarios en el canal de WhatsApp del grupo con el formato: ¿Qué hice ayer? ¿Qué haré hoy? ¿Hay algún bloqueo?
- **Sprint Review:** cada avance ante el docente (13 y 20 de marzo) funcionó como revisión formal del sprint.
- **Retrospectiva:** al final de cada semana identificamos un punto de mejora concreto para el siguiente sprint.

El tablero de tareas se gestionó en **GitHub Projects** con columnas *Backlog*, *In Progress*, *In Review* y *Done*.

6 Planificación de las 3 Semanas de Desarrollo

Semana	Tarea	Responsable	Estado
1	Definición de arquitectura y tecnologías	Est. 1 & 3	Completado
1	Estructura base HTML/CS-S/JS	Est. 2	Completado
1	Boceto del algoritmo de triaje (pseudocódigo)	Est. 3	Completado
1	Inicio del documento L ^A T _E X	Est. 4	Completado
1	Configuración de GitHub y ramas	Est. 5	Completado
2	Mapa interactivo con nodos simulados	Est. 2	Completado
2	Motor de triaje v1 (puntuación básica)	Est. 3	Completado
2	Integración mapa + motor de triaje	Est. 1 & 2	Completado
2	Secciones 1–4 del documento	Est. 4	Completado
2	Pruebas funcionales básicas	Est. 5	Completado

Semana	Tarea	Responsable	Estado
3	Motor de triaje v2 (múltiples recursos)	Est. 3	Completado
3	Panel de control y estadísticas	Est. 2	Completado
3	Secciones 5–10 del documento	Est. 4	Completado
3	Revisión integral y pruebas de estrés	Est. 1 & 5	Completado
3	Preparación de la exposición	Todos	Completado

6.1 Diagrama de Gantt simplificado

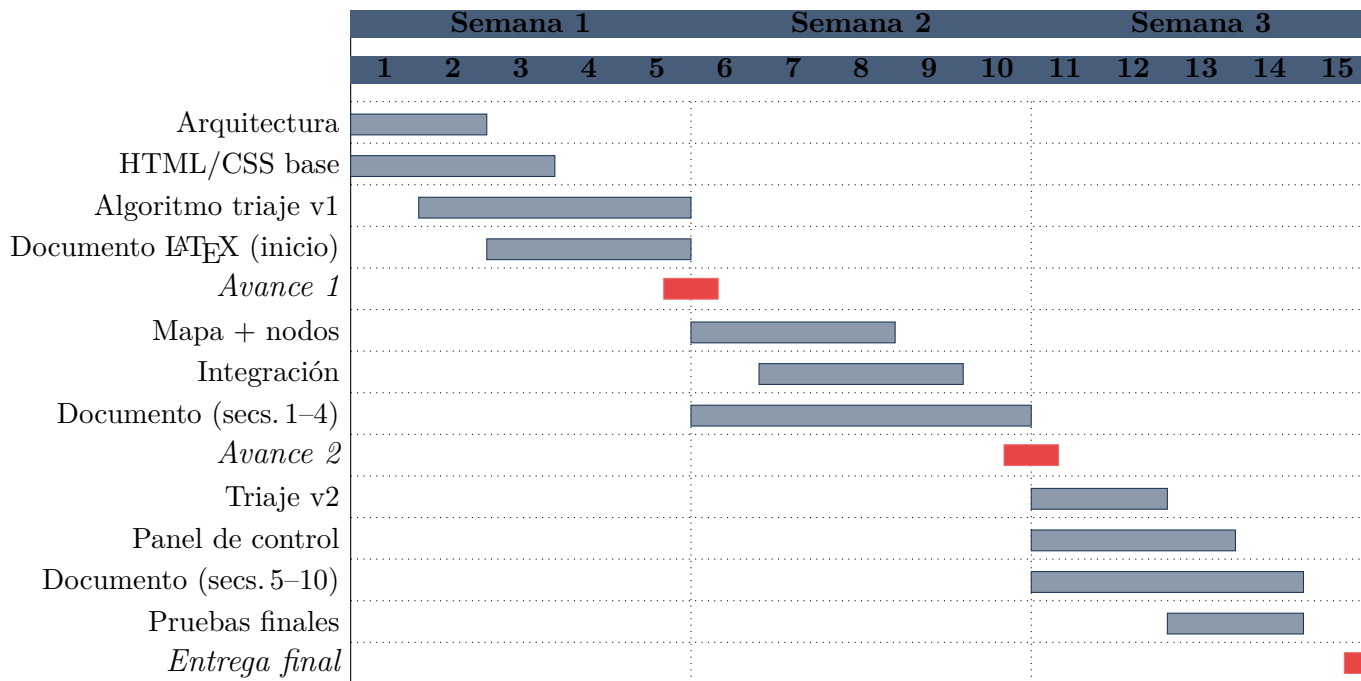


Figura 2: Planificación Gantt del proyecto ResQ-Net.

7 Componente Ético

7.1 El dilema del triaje algorítmico

ResQ-Net debe resolver un problema que ningún sistema técnico puede eludir: *¿a quién enviar el último kit de medicamentos cuando hay más personas que recursos?* Esta pregunta tiene dimensiones étnicas, socioeconómicas y médicas que un algoritmo simplifica necesariamente.

La tentación de diseñar un sistema que “maximice vidas salvadas” esconde supuestos filosóficos con consecuencias concretas: ¿se priorizan los menores de edad? ¿Se descarta

a personas mayores de cierta edad? ¿Cómo se maneja la incertidumbre en el diagnóstico de campo?

Nuestra posición: el algoritmo de triaje de ResQ-Net no toma decisiones autónomas finales. Produce **recomendaciones priorizadas con criterios explícitos y auditables** que siempre deben ser confirmadas por un coordinador humano. La última palabra es siempre humana.

7.2 Aplicación del Código de Ética IEEE/ACM

Aplicamos principalmente el **Principio 1 del ACM Code of Ethics**: “*Un ingeniero de software actuará de manera consistente con el interés público*” [6], concretado en las siguientes decisiones de diseño:

- **Transparencia algorítmica:** cada recomendación muestra los criterios usados (gravedad médica, distancia, stock disponible) para que el coordinador pueda cuestionarla.
- **Privacidad por defecto:** los datos de víctimas no contienen nombres propios ni identificadores personales; solo coordenadas, gravedad y tipo de necesidad.
- **Equidad:** el algoritmo no incluye variables socioeconómicas, étnicas ni de género. Solo criterios médicos y logísticos.
- **Trazabilidad:** cada decisión de asignación queda registrada con timestamp para auditoría post-crisis.

También aplicamos el **Principio 3 (Producto)**: el MVP debe ser seguro incluso en condiciones extremas. Por eso priorizamos la resiliencia sobre la funcionalidad avanzada: es preferible un sistema que funcione sin internet pero que pierda algunas características, a uno que falle por completo cuando la conectividad no está disponible.

7.3 Limitaciones éticas reconocidas

- El sistema depende de la calidad de los datos ingresados en campo (“garbage in, garbage out”). La fatiga y el estrés de las brigadas pueden introducir errores.
- La simulación del MVP no prueba el sistema bajo condiciones reales de caos, latencia de red o falla de hardware.
- El sesgo en los datos de entrenamiento (si se usa ML en versiones futuras) podría reproducir desigualdades históricas en la respuesta humanitaria.

8 Impacto Social Esperado

8.1 Beneficiarios

- **Directos:** personas afectadas por desastres que reciben atención más rápida y coordinada; brigadas de rescate que operan con mejor información; coordinadores

que toman decisiones con datos en tiempo real.

- **Indirectos:** ONGs y organismos estatales que analizan los datos post-crisis para mejorar protocolos futuros; aseguradoras y gobiernos que reducen costos de operaciones mal coordinadas.

8.2 Indicadores de impacto

Indicador	Línea base	Meta (6 meses)
Tiempo promedio de primera respuesta coordinada	4–6 h	< 1 h
Porcentaje de duplicación de entregas	30–40 %	< 5 %
Cobertura de zonas sin asistencia	Sin métrica	< 10 % sin visita
Decisiones de triaje auditadas	0 %	100 %

8.3 Escalabilidad

La arquitectura Mesh permite incorporar nodos nuevos de forma orgánica: basta con conectar un dispositivo a la red local del sector para que empiece a sincronizar datos. No hay un servidor central que pueda convertirse en cuello de botella. En una versión de producción, los nodos fog podrían ser mochilas con Raspberry Pi y antenas de largo alcance (LoRa o Wi-Fi 5 GHz), desplegadas desde helicóptero en los primeros minutos del desastre.

9 Conclusiones y Trabajo Futuro

9.1 Lecciones aprendidas

Desarrollar ResQ-Net en tres semanas dejó lecciones que difícilmente habríamos aprendido en el aula. La más importante: la arquitectura importa más que las features. Dedicar la primera semana a diseñar la topología de nodos y el protocolo de sincronización fue la decisión que salvó el proyecto; cada funcionalidad que construimos después encajó sin fricciones estructurales.

Otra lección fue la dificultad de diseñar algoritmos que tomen decisiones con consecuencias humanas reales. El debate interno sobre qué variables incluir en el modelo de triaje —y cuáles excluir para evitar discriminación— fue más útil que cualquier lectura sobre ética de IA.

9.2 Limitaciones del MVP

- La red Mesh está simulada en el navegador; no probamos latencias ni pérdidas de paquetes reales.
- El mapa usa datos estáticos; en producción necesitaría actualizarse con imágenes satelitales post-desastre.
- El motor de triaje es determinista; en escenarios reales, la incertidumbre médica requeriría modelos probabilísticos.

9.3 Trabajo futuro

1. **v2.0:** implementar la red Mesh real sobre dispositivos Android con Wi-Fi Direct y Bluetooth LE.
2. **v2.1:** integrar imágenes de drones en tiempo real para actualizar el mapa automáticamente.
3. **v3.0:** añadir un modelo de ML para predicción de demanda de recursos basado en el tipo y magnitud del desastre.
4. **v3.1:** integración con sistemas de la UNGRD y la Cruz Roja Colombiana para pruebas piloto en simulacros reales.

9 Referencias

9 Referencias

- [1] United Nations Office for the Coordination of Humanitarian Affairs (OCHA). *Global Humanitarian Overview 2023*. Ginebra: OCHA, 2023. [unocha.org/global-humanitarian-overview-2023](https://www.unocha.org/global-humanitarian-overview-2023)
- [2] Unidad Nacional para la Gestión del Riesgo de Desastres (UNGRD). *Informe anual de gestión del riesgo 2022*. Bogotá: UNGRD, 2022. portal.gestiondelriesgo.gov.co
- [3] World Health Organization (WHO). *Emergency Response Framework*, 2.^a ed. Ginebra: WHO, 2022. [who.int/publications](https://www.who.int/publications)
- [4] M. Schuller y P. Morales (eds.). *Tectonic Shifts: Haiti Since the Earthquake*. Sterling: Kumarian Press, 2012.
- [5] H. Plattner. *An Introduction to Design Thinking: Process Guide*. Stanford: Hasso Plattner Institute of Design at Stanford University, 2010.
- [6] ACM & IEEE-CS Joint Task Force on Software Engineering Ethics. *Software Engineering Code of Ethics and Professional Practice*, v5.2. Nueva York: ACM, 2018. [acm.org/code-of-ethics](https://www.acm.org/code-of-ethics)

- [7] A. S. Tanenbaum y D. J. Wetherall. *Computer Networks*, 5.^a ed. Upper Saddle River: Pearson, 2011.
- [8] F. Bonomi, R. Milito, J. Zhu y S. Addepalli. “Fog Computing and its Role in the Internet of Things”. En: *Proc. of the First Edition of the MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [9] I. Lee y K. Lee. “The Internet of Things (IoT): Applications, investments, and challenges for enterprises”. *Business Horizons*, vol. 58, n.º 4, pp. 431–440, 2015.
- [10] OpenStreetMap contributors. *OpenStreetMap* [en línea]. Disponible en: openstreetmap.org [Acceso: marzo 2026].